SDEV 2401

Rapid Backend App Development

REST API Development - 3

A LEADING POLYTECHNIC COMMITTED TO YOUR SUCCESS

Expectations - What I expect from you

- No Late Assignments

- No Cheating

- Be a good classmate

- Don’t waste your time

- Show up to class

WE ARE ESSENTIAL TO ALBERTA | NAIT

Agenda

On the right is what we will cover today.

- ModelSerializers in DRF
- Routers and URL Patterns
- Permissions and Access Control
- Token-Based Authentication
- Obtaining and Using Tokens
- Other Token Types
- Summary

WE ARE ESSENTIAL TO ALBERTA
NAIT

ModelSerializers in DRF

A `ModelSerializer` in Django REST Framework (DRF) automatically generates serializer fields based on a model, similar to how a `ModelForm` works.

Suppose you have a `Book` model:

```
from rest_framework import serializers
from .models import Book

class BookSerializer(serializers.ModelSerializer):
    class Meta:
        model = Book
        fields = '__all__'
```

- This serializer will include all fields from the `Book` model.
- If you add new fields to `Book`, they are automatically included.
- You can specify fields explicitly if you want more control.

WE ARE ESSENTIAL TO ALBERTA
NAIT

Routers and URL Patterns

DRF routers can automatically generate URL patterns for your viewsets, reducing boilerplate code.

Registering a `BookViewSet` with a router:

```
from rest_framework.routers import DefaultRouter
from .views import BookViewSet

router = DefaultRouter()
router.register(r'books', BookViewSet)

urlpatterns = [
    # ...other patterns...
] + router.urls
```

- The router creates endpoints like `/books/` and `/books/{id}/`.
- This approach avoids manual URL pattern definitions for each CRUD operation.
- You can still add custom endpoints alongside router-generated ones.

WE ARE ESSENTIAL TO ALBERTA
NAIT

Permissions and Access Control

DRF permissions control who can access your API endpoints.

Allowing only authenticated users to access a viewset:

```
from rest_framework.permissions import IsAuthenticated

class BookViewSet(viewsets.ModelViewSet):
    permission_classes = [IsAuthenticated]
    queryset = Book.objects.all()
    serializer_class = BookSerializer
```

- `IsAuthenticated` restricts access to users who are logged in.
- You can use other permission classes for different access rules.
- Permissions can be set globally or per-view.

WE ARE ESSENTIAL TO ALBERTA
NAIT

Token-Based Authentication

Token authentication allows clients to authenticate using a token instead of a username and password.

Setting up token authentication in `settings.py`:

```
INSTALLED_APPS = [
    # ... other apps ...
    'rest_framework.authtoken',
]

REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': [
        'rest_framework.authentication.TokenAuthentication',
    ],
}
```

- Tokens are generated per user and sent in the request header.
- This is useful for mobile apps or third-party clients.
- Tokens can be managed via the admin panel or programmatically.

WE ARE ESSENTIAL TO ALBERTA
NAIT

Obtaining and Using Tokens

Clients obtain tokens by sending credentials to a special endpoint, then use the token for future requests.

Adding a token endpoint to your URLs:

```
from rest_framework.authtoken.views import obtain_auth_token

urlpatterns = [
    # ... other patterns ...
    path('api/token-auth/', obtain_auth_token, name='api_token_auth'),
]
```

- Clients POST their username and password to `/api/token-auth/` to get a token.
- Future requests include `Authorization: Token <token>` in the header.
- Without a valid token, protected endpoints return a permission denied error.

WE ARE ESSENTIAL TO ALBERTA
NAIT

Other Token Types

Beyond DRF's default tokens, you can use JWT or OAuth2 for more advanced authentication.

JWT header example:

```
Authorization: Bearer <your_jwt_token>
```

- JWT and OAuth2 support features like token expiration and refresh.
- Use packages like `djangorestframework-simplejwt` or `django-oauth-toolkit` for these methods.
- Choose the token type that fits your application's security needs.

WE ARE ESSENTIAL TO ALBERTA
NAIT

Summary

- DRF provides powerful tools for building REST APIs with Django.
- ModelSerializers simplify data serialization.
- ModelViewSets and routers reduce the amount of code needed for CRUD operations.
- Routers reduce URL configuration overhead.
- Permissions and token authentication enhance security.
- Choose the right authentication method based on your application’s requirements.

Example

Let’s go do an example together